

Практическая работа №1. Введение в инструментарий анализа данных

1. Назначение практической работы

Цель работы — освоить полный базовый цикл анализа табличных данных: подготовить воспроизводимое Python-окружение, загрузить данные, проверить их структуру и качество, выполнить предобработку, получить описательные статистики, сформировать простые аналитические срезы, визуализировать закономерности и сохранить подготовленный набор данных в обменных и аналитических форматах.

В отличие от работы, построенной вокруг отдельных конструкций языка Python, здесь синтаксис используется только как инструмент решения аналитической задачи. Каждая операция должна сопровождаться интерпретацией результата. Студент отвечает не только на вопрос «что вернула функция?», но и на вопрос «что это означает для исследуемого набора данных?».

Ожидаемые результаты обучения

- создавать изолированное окружение Python и подключать библиотеки анализа данных;
- объяснять различия между CSV, JSON и Parquet с точки зрения структуры и аналитической обработки;
- использовать DataFrame и Series, понимать типы данных и влияние dtype на память;
- применять head(), info(), describe(), isna(), duplicated(), nunique(), loc(), query(), groupby() и agg();
- выполнять векторные преобразования и создавать вычисляемые признаки;
- проводить базовый EDA: распределения, частоты, групповые статистики, корреляции и поиск потенциальных выбросов;
- выбирать базовый тип визуализации под аналитический вопрос;
- экспортировать подготовленные данные и формулировать воспроизводимые выводы.

Исходные данные

Файл FiresRu.csv¹ содержит 26 681 наблюдений и 9 признаков. В нём представлены категория события, географические координаты и набор метеорологических характеристик. В файле отсутствует поле даты/времени,

¹ Метеорологические условия пожаров на территории России (2021–2022) <https://data.rcsi.science/data-catalog/datasets/209/>, дата обновления: 24.04.2026

поэтому временной ряд по этим данным построить нельзя без внешнего источника временной информации.

Таблица 1. Описание набора данных пожаров в России

Поле	Тип	Интерпретация
<i>type_name</i>	object / category	Тип события; в файле 5 категорий.
<i>type_id</i>	int	Числовой код типа события; аналитически это категориальный код, а не непрерывная величина.
<i>lon</i>	float	Долгота.
<i>lat</i>	float	Широта.
<i>temperature_c</i>	float	Температура, °С.
<i>precipitation_mm</i>	float	Количество осадков, мм.
<i>relative_humidity</i>	float	Относительная влажность. Единица явно не задана метаданными файла.
<i>wind_speed_ms</i>	float	Скорость ветра, м/с.
<i>solar_radiation</i>	float	Показатель солнечной радиации; единица измерения в CSV не указана.

Нельзя приписывать столбцу физическую единицу, которой нет в метаданных. Например, числовой диапазон *relative_humidity* выглядит как процентная шкала, но формальный отчёт должен отделять наблюдение по данным от утверждения о единицах измерения. Для *solar_radiation* единица в файле вовсе отсутствует.

2. Рабочее окружение Python

Изолированное виртуальное окружение фиксирует набор библиотек конкретного проекта и снижает риск конфликтов версий. Для курса целесообразно использовать один базовый интерпретатор, например Python 3.11, чтобы

последующие работы с PySpark выполнялись в том же окружении. Jupyter Notebook удобен для интерактивного исследования, а VS Code позволяет выполнять тот же код в интерактивном режиме или скриптовом режиме.

Пакеты устанавливаются внутрь активированного окружения. Для этой работы нужны pandas и numpy для вычислений, matplotlib для графиков, jupyter для интерактивной работы и pyarrow для Parquet.

Практический блок 2.1. Создание окружения

В этом блоке используются команды оболочки и механизм запуска модулей Python. Команда `py -3.11` выбирает интерпретатор Python 3.11, ключ `-m` запускает указанный модуль как программу, а модуль `venv` создаёт изолированное окружение в каталоге `.venv`. После активации окружения конструкция `python -m pip` гарантирует, что менеджер пакетов относится именно к текущему интерпретатору. Ключ `--upgrade` обновляет `pip`, а перечисленные после `install` имена задают устанавливаемые пакеты. Команда `jupyter notebook` запускает локальный сервер Jupyter и открывает интерфейс Notebook в браузере.

```
# Windows PowerShell / cmd
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install pandas numpy matplotlib pyarrow jupyter

# Запуск Jupyter
jupyter notebook
```

После установки команды `import pandas`, `import numpy` и `import matplotlib` должны выполняться без `ImportError`. Номера версий будут зависеть от даты установки и поэтому в методичке не фиксируются.

Проверка окружения

```
import sys
import pandas as pd
import numpy as np
import matplotlib

print("Python:", sys.version.split()[0])
print("pandas:", pd.__version__)
print("NumPy:", np.__version__)
print("Matplotlib:", matplotlib.__version__)
```

Окружение считается подготовленным, если интерпретатор запускается из `.venv`, библиотеки импортируются без ошибок, а `notebook` выполняет ячейки последовательно. Это делает практику воспроизводимой и отделяет зависимости курса от системной установки Python.

3. Минимальное введение в Python, NumPy и Pandas для анализа

Python хранит значения в объектах с динамическими типами, однако для анализа данных критично не столько объявление переменных, сколько структура массива и тип элементов. NumPy предоставляет однородные многомерные массивы `ndarray` и векторные операции, а Pandas — `Series` и `DataFrame` с индексами, именованными столбцами и смешанными типами данных.

Главный практический принцип — по возможности заменять построчные циклы векторными выражениями. Операция над целым столбцом читается как аналитическое правило и обычно эффективнее Python-цикла.

Практический блок 3.1. Векторное преобразование

В следующем фрагменте будет смоделирован типичный для анализа данных переход от исходной единицы измерения к производной. Три значения скорости ветра в м/с объединяются в массив, затем весь массив одним выражением переводится в км/ч по формуле $v_{\text{km/h}} = 3.6 \cdot v_{\text{m/s}}$. Функция `np.round(array, 2)` округляет все элементы результата до двух знаков после десятичной точки.

```
import numpy as np

wind_ms = np.array([1.14, 0.80, 0.97])
wind_kmh = wind_ms * 3.6
print(np.round(wind_kmh, 2))
```

Результат выполнения

```
[4.10 2.88 3.49]
```

Умножение выполняется сразу для всего массива. Точно тот же принцип используется ниже для столбца `DataFrame`: `df["wind_speed_ms"] * 3.6`.

Практический блок 3.2. Ключевые объекты Pandas

Сначала создадим минимальные примеры `Series` и `DataFrame`, после чего программа явно покажет их типы. Далее сгруппируем данные по типам и подсчитаем статистику.

`pd.Series()` создаёт одномерную индексированную структуру Pandas, а параметр `name` задаёт имя серии.

`pd.DataFrame()` формирует таблицу, в данном примере ей передаётся словарь, где ключи становятся именами столбцов, а списки — их значениями.

Цепочка `frame.groupby("type")["value"].sum()` сначала группирует строки по значениям столбца `type`, затем выбирает столбец `value` и вычисляет сумму внутри каждой группы.

```
s = pd.Series([10, 20, 30], name="value")
frame = pd.DataFrame({
    "type": ["A", "B", "A"],
    "value": [10, 20, 30]
})

print(type(s).__name__)
print(type(frame).__name__)
print(frame.groupby("type")["value"].sum())
```

Результат выполнения

```
Series
DataFrame
type
A      40
B      20
Name: value, dtype: int64
```

`Series` — один именованный столбец с индексом; `DataFrame` — набор согласованных `Series`. `groupby()` реализует типичную аналитическую операцию «разбить на группы → агрегировать».

4. Форматы файлового хранения данных и загрузка CSV

CSV — текстовый табличный формат без строгой схемы типов: значения хранятся строками, а типы восстанавливаются при чтении. JSON удобен для вложенных и полуструктурированных объектов, но при записи табличных данных часто занимает больше места. Parquet — бинарный колоночный формат, сохраняющий типы и ориентированный на аналитическое чтение отдельных столбцов. В последующих работах Spark будет особенно эффективно использовать Parquet.

Таблица 2. Популярные форматы файлового хранения данных

Формат	Организация	Схема	Сильная сторона	Ограничение
CSV	Строки/разделители	Нет	Высокая	Повторное

Формат	Организация	Схема	Сильная сторона	Ограничение
		встроенной строгой схемы	совместимость, просмотр текстом	определение типов; чтение лишних столбцов.
JSON	Объекты/массивы	Типы выражены структурой JSON	Гибкая вложенная структура	Избыточность ключей и большой размер для таблиц.
Parquet	Колоночный бинарный	Да	Сжатие, сохранение типов, выборочное чтение столбцов	Нужен специальный reader/engine.

Практический блок 4.1. Загрузка FiresRu.csv

В блоке выполним первичную проверку самого факта корректной загрузки данных. Сначала создадим путь к FiresRu.csv, затем считаем содержимое файла в объект df. После чтения проверим размер таблицы и состав полей, и с помощью head() покажем несколько реальных наблюдений.

Класс Path из стандартного модуля pathlib представляет путь к файлу как объект и позволяет не связывать код с особенностями строковых путей конкретной операционной системы.

Функция pd.read_csv() читает CSV и возвращает DataFrame. Атрибут df.shape содержит кортеж пары (число строк, число столбцов), df.columns — индекс с именами столбцов, а .tolist() преобразует его в обычный список Python.

Метод head() возвращает первые пять строк и используется для быстрой визуальной проверки структуры загруженной таблицы.

Задача состоит в том, чтобы убедиться, что файл прочитан как ожидаемая таблица и что дальнейший анализ будет выполняться над правильным объектом.

```
from pathlib import Path
import pandas as pd

DATA_PATH = Path("FiresRu.csv")
df = pd.read_csv(DATA_PATH)
```

```
print(df.shape)
print(df.columns.tolist())
df.head()
```

Результат выполнения

```
(26681, 9)
['type_name', 'type_id', 'lon', 'lat', 'temperature_c',
'precipitation_mm', 'relative_humidity', 'wind_speed_ms',
'solar_radiation']
```

Таблица 3. Результат выполнения блока 4.1. в виде таблицы

type_name	type_id	lon	lat	temperature_c	precipitation_mm	relative_humidity	wind_speed_ms	solar_radiation
Natural fire	4	131.59	47.87	-9.44	0.01	72.98	1.14	4.59
Natural fire	4	131.59	47.88	-9.44	0.01	72.98	1.14	4.59
Forest fire	3	131.99	48.50	-10.71	0.05	81.31	0.80	4.24
Natural fire	4	131.90	43.63	-7.77	0.00	68.41	0.97	4.66
Natural fire	4	131.57	47.86	-9.44	0.01	72.98	1.14	4.59

Загрузка прошла без явного задания `sep` и `encoding`: файл использует стандартный разделитель «`,`» и читается как таблица из 26 681 строк и 9 столбцов. По перечню полей сразу видно отсутствие временной метки — значит, временной анализ на этом наборе не является корректной задачей.

5. Первичная диагностика данных

Первичный анализ до построения графиков и вычисления сложных метрик ставит своей целью выяснить какой размер у входной таблицы данных, какие типы данных в атрибутах распознаны, есть ли пропуски и дубликаты в данных, какие диапазоны значений наблюдаются.

Метод `head()` показывает примеры строк, `info()` — структуру и память, `describe()` — основные статистики, `isna()` и `duplicated()` — простейшие индикаторы качества.

Практический блок 5.1. Структура и память

Вместо непосредственного вывода `info()` в интерфейс Notebook его результат будет сначала перехвачен в строковый буфер, а затем напечатан. Это удобно для воспроизводимого отчёта и одновременно позволяет изучить структуру таблицы и фактическое потребление памяти. По результату необходимо обратить внимание не только на отсутствие пропусков, но и на тип `object` у категориального

столбца `type_name`, поскольку именно он далее станет кандидатом на оптимизацию.

Модуль `io` входит в стандартную библиотеку Python. Объект `io.StringIO()` создаёт текстовый поток в оперативной памяти и может использоваться как временный «файл».

Метод `df.info()` выводит сводку о `DataFrame`: тип индекса, число строк, имена столбцов, количество непустых значений и `dtypes`. Параметр `buf` перенаправляет этот вывод в созданный поток, а `memory_usage="deep"` просит Pandas глубже учитывать память, занятую объектными значениями. Метод `getvalue()` извлекает накопленный текст из `StringIO`.

```
import io

buf = io.StringIO()
df.info(buf=buf, memory_usage="deep")
print(buf.getvalue())
```

Результат выполнения

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26681 entries, 0 to 26680
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   type_name              26681 non-null  object
1   type_id                26681 non-null  int64
2   lon                    26681 non-null  float64
3   lat                    26681 non-null  float64
4   temperature_c          26681 non-null  float64
5   precipitation_mm        26681 non-null  float64
6   relative_humidity       26681 non-null  float64
7   wind_speed_ms           26681 non-null  float64
8   solar_radiation         26681 non-null  float64
dtypes: float64(7), int64(1), object(1)
memory usage: 3.2 MB
```

Тип `object` у `type_name` означает строковые Python-объекты. Для столбца с пятью повторяющимися категориями это функционально корректно, но избыточно по памяти. Ниже он будет преобразован в `category`.

Практический блок 5.2. Пропуски, дубликаты и уникальные значения

Следующий код выполняет три независимые проверки качества данных: полноту, наличие полных дублей и кардинальность признаков. Сначала будет подсчитано число пропусков по каждому полю, затем — число полностью совпадающих строк, после чего для всех столбцов определяется количество уникальных значений. Кардинальность особенно важна для распознавания категориальных признаков и для обнаружения полей, которые внешне имеют числовой `dtype`, но фактически являются кодами категорий.

Метод `isna()` строит `DataFrame` логических значений: `True` соответствует пропуску, `False` — заполненному элементу.

Последующий `sum()` суммирует `True` как единицы и тем самым даёт число пропусков по каждому столбцу.

`duplicated()` возвращает булеву маску полных повторов строк; её `sum()` считает количество таких повторов.

Метод `nunique()` вычисляет число различных значений отдельно для каждого столбца.

```
print("Пропуски по столбцам:")
print(df.isna().sum())
print("Полных дубликатов:", df.duplicated().sum())
print("Число уникальных значений:")
print(df.nunique())
```

Результат выполнения

```
Пропуски по столбцам:
type_name      0
type_id        0
lon             0
lat            0
temperature_c   0
precipitation_mm 0
relative_humidity 0
wind_speed_ms   0
solar_radiation 0
Полных дубликатов: 0
Число уникальных значений:
type_name      5
type_id        5
lon           25708
lat           23547
temperature_c   2581
precipitation_mm  571
relative_humidity 4028
wind_speed_ms    532
solar_radiation   731
```

В данных нет пропущенных значений и нет полностью совпадающих строк. Это не отменяет семантической проверки. Например, `type_id` необходимо интерпретировать как код категории, а не как величину, для которой осмысленно среднее арифметическое.

Практический блок 5.3. Проверка соответствия кода и категории

Задача этого блока — проверить семантическую согласованность двух представлений одного и того же признака числового `type_id` и текстового `type_name`. Из исходной таблицы будут выделены только эти поля,

повторяющиеся пары удалены, а уникальные соответствия отсортированы по коду. Если одному коду соответствовало бы несколько названий или одному названию — несколько кодов, это стало бы заметно в полученной таблице и потребовало бы дополнительной проверки качества данных.

Выражение `df[["type_id", "type_name"]]` выбирает из `DataFrame` два столбца и возвращает новый `DataFrame`.

Метод `drop_duplicates()` удаляет повторяющиеся пары значений, а `sort_values("type_id")` упорядочивает оставшиеся строки по числовому коду.

Круглые скобки позволяют записать цепочку методов на нескольких строках без символа продолжения.

Метод `to_string(index=False)` формирует текстовое представление таблицы без вывода индекса.

```
type_map = (
    df[["type_id", "type_name"]]
      .drop_duplicates()
      .sort_values("type_id")
)
print(type_map.to_string(index=False))
```

Таблица 4. Результат выполнения блока 5.3 в виде таблицы

type_id	type_name
1	Uncontrolled burn
2	Peat fire
3	Forest fire
4	Natural fire
5	Controlled burn

Набор содержит взаимно однозначное соответствие пяти кодов пяти названиям событий, среди которых 1 — Uncontrolled burn, 2 — Peat fire, 3 — Forest fire, 4 — Natural fire, 5 — Controlled burn. Следовательно, `type_id` можно хранить компактным целым типом, но в статистическом анализе рассматривать как категориальный признак.

Практический блок 5.4. Описательная статистика

Следующий код формирует компактную статистическую сводку для координат и метеорологических полей. По ней нужно оценивать центральные значения, разброс и границы диапазона, а также искать признаки, у которых медиана

заметно отличается от среднего или максимум сильно удалён от верхнего квартиля.

Список `cols` задаёт явный набор количественных признаков, которые должны участвовать в статистическом описании. Выражение `df[cols]` выбирает только эти столбцы.

Метод `describe()` для числовых данных рассчитывает `count`, `mean`, `std`, `minimum`, квартиль 25%, медиану 50%, квартиль 75% и `maximum`. Метод `round(2)` округляет результат до двух знаков, а свойство `.T` транспонирует таблицу, превращая признаки в строки — такой формат удобнее для чтения широкого набора показателей.

В статистику намеренно не включается `type_id`, хотя технически он имеет целочисленный `dtype`. Это код категории, и среднее значение для него не имеет содержательного смысла.

```
cols = [
    "lon", "lat", "temperature_c", "precipitation_mm",
    "relative_humidity", "wind_speed_ms", "solar_radiation"
]
print(df[cols].describe().round(2).T)
```

Таблица 5. Результат выполнения практического блока 5.4 в виде таблицы

Признак	count	mean	std	min	25%	50%	75%	max
<i>lon</i>	26681.00	106.35	27.31	20.37	82.41	96.56	133.90	175.87
<i>lat</i>	26681.00	57.53	4.30	41.71	53.95	58.97	60.64	70.33
<i>temperature_c</i>	26681.00	17.89	5.68	-34.88	15.70	19.08	21.39	31.94
<i>precipitation_m m</i>	26681.00	0.46	1.46	0.00	0.00	0.04	0.30	25.85
<i>relative_humidit y</i>	26681.00	69.44	11.43	24.70	62.93	71.35	77.91	99.83
<i>wind_speed_ms</i>	26681.00	0.63	0.93	0.00	0.11	0.25	0.66	7.42
<i>solar_radiation</i>	26681.00	5.57	1.50	0.42	4.55	5.68	6.71	8.97

Температура изменяется от $-34,88$ до $31,94$ °C; медиана равна $19,08$ °C. Для `precipitation_mm` медиана составляет $0,04$ мм при максимуме $25,85$ мм, а для `wind_speed_ms` медиана $0,25$ м/с при максимуме $7,42$ м/с. Такие соотношения уже указывают на асимметричность некоторых распределений и необходимость визуальной проверки.

6. Типы данных и предобработка

Предобработка — это не обязательное исправление каждого столбца, а приведение таблицы к типам и представлениям, соответствующим смыслу данных. Категории целесообразно хранить как `category`, небольшие целочисленные коды — в компактных целых типах, числовые преобразования выполнять векторно. Удалять строки следует только при наличии обоснованной причины.

В данном файле пропусков и полных дубликатов нет, поэтому очистка не должна искусственно удалять наблюдения. Вместо этого полезно оптимизировать `dtypes` и создать вычисляемые признаки, не меняя исходную информацию.

Практический блок 6.1. Оптимизация типов

Сначала будет измерен объём исходного `DataFrame` в памяти. Затем создаётся `clean_df`, в котором два столбца получают типы, соответствующие их смыслу и диапазону значений.

После преобразования память измеряется повторно, а код вычисляет относительное снижение. Такой эксперимент показывает, что предобработка включает не только удаление ошибок, но и приведение представления данных к более эффективной схеме без потери информации.

Метод `memory_usage(deep=True)` возвращает оценку памяти по компонентам `DataFrame`, а `sum()` суммирует её до общего объёма. `df.copy()` создаёт независимую копию таблицы, чтобы не изменять исходный объект.

Метод `astype()` явно преобразует `dtype`. Тип `category` хранит повторяющиеся текстовые категории через компактные коды, а `int8` использует 8-битное целое для небольших числовых кодов. `f`-строки позволяют вставлять вычисляемые выражения в текст. Спецификаторы `:.2f` и `:.1f` задают число знаков после точки.

```
before = df.memory_usage(deep=True).sum()

clean_df = df.copy()
clean_df["type_name"] = clean_df["type_name"].astype("category")
clean_df["type_id"] = clean_df["type_id"].astype("int8")

after = clean_df.memory_usage(deep=True).sum()
print(f"До: {before / 1024**2:.2f} MiB")
print(f"После: {after / 1024**2:.2f} MiB")
print(f"Снижение: {(1 - after / before) * 100:.1f}%")
```

Результат выполнения

```
До: 3.18 MiB
После: 1.48 MiB
Снижение: 53.5%
```

Преобразование `type_name` в `category` особенно эффективно, потому что вместо 26 681 повторяющейся строки Pandas хранит компактные коды категорий и таблицу уникальных значений.

Практический блок 6.2. Создание вычисляемых признаков

В таблицу будут добавлены два разных по смыслу производных признака. `wind_speed_kmh` является преобразованием единиц измерения и не добавляет новой физической информации, тогда как `is_forest_fire` кодирует принадлежность строки к одной категории в бинарном виде.

После создания столбцов выводятся исходная скорость, преобразованная скорость и логический индикатор, чтобы визуально проверить корректность вычислений на первых наблюдениях.

Присваивание вида `clean_df["new_column"] = expression` создаёт новый столбец либо заменяет существующий.

Арифметика над Series выполняется векторно. Умножение `wind_speed_ms` на 3.6 применяется ко всем строкам, после чего `round(2)` округляет значения.

Сравнение Series со строкой "Forest fire" также выполняется поэлементно и возвращает логическую Series из True/False.

Выражение `clean_df[...].head()` выбирает несколько полей и показывает первые пять строк результата.

```
clean_df["wind_speed_kmh"] = (
    clean_df["wind_speed_ms"] * 3.6
).round(2)

clean_df["is_forest_fire"] = (
    clean_df["type_name"] == "Forest fire"
)

print(clean_df[
    ["type_name", "wind_speed_ms", "wind_speed_kmh",
    "is_forest_fire"]
].head())
```

Таблица 6. Результат выполнения блока 6.2. в виде таблицы

type_name	wind_speed_ms	wind_speed_kmh	is_forest_fire
<i>Natural fire</i>	1.14	4.10	False
<i>Natural fire</i>	1.14	4.10	False
<i>Forest fire</i>	0.80	2.88	True
<i>Natural fire</i>	0.97	3.49	False
<i>Natural fire</i>	1.14	4.10	False

`wind_speed_kmh` — линейное преобразование единиц, а `is_forest_fire` — логический индикатор категории. Первый столбец не добавляет новой физической информации, но удобен для представления. Второй пригоден для фильтрации и может выступать бинарной меткой в последующих задачах.

7. Фильтрация, группировка и агрегирование

Фильтрация отвечает на вопрос «какие строки удовлетворяют условию?», а группировка — «как отличается агрегат между категориями?». В Pandas булево выражение формирует маску длины N, после чего `loc` выбирает строки и столбцы. Метод `groupby()` делит данные на группы, а `agg()` вычисляет несколько статистик за один проход логической операции.

Практический блок 7.1. Распределение наблюдений по типам

В следующем фрагменте будет построена таблица распределения классов: для каждого `type_name` определяются и число наблюдений, и его доля в процентах. Результат нужно интерпретировать не только как рейтинг частот, но и как диагностику баланса выборки.

Метод `value_counts()` подсчитывает частоту каждого уникального значения Series и по умолчанию сортирует категории по убыванию частоты.

`to_frame("n")` преобразует полученную Series в DataFrame со столбцом `n`.

Метод `assign()` добавляет вычисляемый столбец и возвращает новый DataFrame; в данном случае значение `share_pct` задаётся через `lambda` — небольшую анонимную функцию.

`len(df)` возвращает общее число строк, поэтому `n / len(df) * 100` переводит абсолютные частоты в проценты.

```
type_stats = (
    df["type_name"]
      .value_counts()
      .to_frame("n")
      .assign(share_pct=lambda x: x["n"] / len(df) * 100)
)
print(type_stats.round(2))
```

Таблица 7. Результат выполнения блока 7.1 в виде таблицы

type_name	n	share_pct
<i>Forest fire</i>	18415	69.02
<i>Natural fire</i>	5195	19.47
<i>Uncontrolled burn</i>	2711	10.16

type_name	n	share_pct
<i>Controlled burn</i>	333	1.25
<i>Peat fire</i>	27	0.10

Категория Forest fire составляет 69.02% всех строк (18 415 наблюдений), тогда как Peat fire — только 0.10% (27 наблюдений). Набор существенно несбалансирован по типу события. Это важно учитывать при сравнении категорий и особенно при будущей классификации.

Практический блок 7.2. Групповые средние

Код сформирует одну сравнительную таблицу, в которой для каждого типа события одновременно вычисляются объём группы и средние значения температуры, осадков, влажности, скорости ветра и солнечной радиации.

Метод `groupby("type_name")` разбивает строки на группы по типу события. Параметр `observed=True` особенно важен для категориальных типов данных поскольку он ограничивает результат реально наблюдавшимися категориями.

Метод `agg()` поддерживает именованные агрегации в форме `новое_имя=(исходный_столбец, функция)`.

Функция `size` считает число строк группы, а `mean` — среднее выбранного признака.

`sort_values("n", ascending=False)` упорядочивает итог по размеру групп от большей к меньшей.

```
group_stats = (
    df.groupby("type_name", observed=True)
      .agg(
          n=("type_name", "size"),
          temp_mean=("temperature_c", "mean"),
          precip_mean=("precipitation_mm", "mean"),
          humidity_mean=("relative_humidity", "mean"),
          wind_mean=("wind_speed_ms", "mean"),
          solar_mean=("solar_radiation", "mean")
      )
      .sort_values("n", ascending=False)
)
print(group_stats.round(2))
```

Таблица 8. Результат выполнения блока 7.2. в виде таблицы

type_name	n	temp_mea n	precip_mea n	humidity_mea n	wind_mea n	solar_mea n
Forest fire	1841	18.69	0.50	70.30	0.42	5.87

type_name	n	temp_mean	precip_mean	humidity_mean	wind_mean	solar_mean
	5					
Natural fire	5195	17.42	0.40	69.07	0.80	5.14
Uncontrolled burn	2711	14.21	0.29	64.35	1.64	4.57
Controlled burn	333	10.78	0.25	69.46	1.46	4.27
Peat fire	27	18.17	0.55	65.47	1.19	5.26

В отчете при анализе результата сначала нужно смотреть на n, а уже затем сравнивать средние. Статистика группы из нескольких десятков наблюдений не должна трактоваться с той же уверенностью, что и статистика группы из тысяч строк. Статистика Peat fire основана только на 27 строках, поэтому прямое сравнение со статистикой для 18415 наблюдений в Forest fire неуместно. Средние описывают конкретный набор наблюдений, но сами по себе не доказывают причинную связь между погодой и типом события.

Практический блок 7.3. Условная фильтрация

Код ниже сформирует срез наблюдений, для которых одновременно выполняются два условия: влажность ниже 40 и скорость ветра выше 2 м/с. Из выбранных строк оставляются только поля, необходимые для интерпретации результата. После фильтрации программа считает объём подвыборки и показывает несколько строк. Индексатор `df.loc[условие, столбцы]` выбирает строки по логической маске и одновременно ограничивает набор возвращаемых столбцов.

Сравнения `df["relative_humidity"] < 40` и `df["wind_speed_ms"] > 2` создают две булевы Series.

Оператор `&` выполняет поэлементное логическое «И»; каждое сравнение заключается в круглые скобки из-за приоритетов операторов Pandas/Python.

Метод `len(subset)` даёт число выбранных строк, а `head()` — первые примеры.

```
subset = df.loc[
    (df["relative_humidity"] < 40) &
    (df["wind_speed_ms"] > 2),
    ["type_name", "lon", "lat", "temperature_c",
     "relative_humidity", "wind_speed_ms"]
]

print("Строк в срезе:", len(subset))
print(subset.head())
```

Результат выполнения

```
Строк в срезе: 337
```

Таблица 9. Результат выполнения блока 7.3 как таблица; результат показывает первые 5 наблюдений из подвыборки

type_name	lon	lat	temperature_c	relative_humidity	wind_speed_ms
Forest fire	91.39	51.06	15.15	27.97	2.55
Forest fire	104.58	50.37	14.67	37.89	2.64
Forest fire	104.49	50.36	14.67	37.89	2.64
Forest fire	104.42	50.33	14.67	37.89	2.64
Forest fire	104.30	50.34	14.67	37.89	2.64

Условия `relative_humidity < 40` и `wind_speed_ms > 2` выбраны только как учебный пример составного фильтра, а не как нормативный критерий пожарной опасности. В наборе им соответствуют 337 строк.

8. Корреляция и потенциальные выбросы

Коэффициент корреляции Пирсона оценивает линейную связь двух числовых признаков и лежит в диапазоне от -1 до 1 . Значение около нуля означает отсутствие выраженной линейной связи, но не отсутствие любой зависимости. Корреляция также не устанавливает причинность.

Для первичного поиска необычных значений можно использовать межквартильный размах $IQR = Q3 - Q1$. Эвристическое правило помечает значения за пределами $[Q1 - 1,5 \cdot IQR; Q3 + 1,5 \cdot IQR]$. Такие наблюдения являются кандидатами на дополнительную проверку, а не автоматически ошибками.

Практический блок 8.1. Корреляционная матрица

Код данного блока построит матрицу попарных линейных связей, после чего нужно анализировать знак и модуль коэффициентов. Цель — определить, какие пары признаков демонстрируют заметную линейную сопряжённость, не превращая корреляцию в утверждение о причинно-следственной зависимости.

В расчёт сознательно включаются только метеорологические показатели: категориальный код `type_id` и географические координаты здесь исключены из рассматриваемой содержательной задачи.

Список `weather` фиксирует набор переменных, между которыми имеет смысл оценивать линейные связи. Выражение `df[weather]` создаёт числовой поднабор `DataFrame`.

Метод `corr()` по умолчанию рассчитывает попарные коэффициенты корреляции Пирсона и возвращает квадратную симметричную матрицу. На диагонали находятся единицы, поскольку каждый признак полностью коррелирует сам с собой. `round(3)` ограничивает вывод тремя знаками после десятичной точки.

```
weather = [
    "temperature_c", "precipitation_mm", "relative_humidity",
    "wind_speed_ms", "solar_radiation"
]

corr = df[weather].corr()
print(corr.round(3))
```

Таблица 10. Корреляционная матрица для атрибутов набора данных

Признак	temperature_c	precipitation_mm	relative_humidity	wind_speed_ms	solar_radiation
temperature_c	1.00	0.00	-0.15	-0.30	0.22
precipitation_mm	0.00	1.00	0.23	-0.05	-0.19
relative_humidity	-0.15	0.23	1.00	-0.28	-0.23
wind_speed_ms	-0.30	-0.05	-0.28	1.00	-0.19
solar_radiation	0.22	-0.19	-0.23	-0.19	1.00

В данном наборе нет очень сильных линейных связей между рассмотренными погодными признаками. Наибольшие по модулю значения среди пар находятся примерно на уровне 0,30: `temperature_c` и `wind_speed_ms` имеют $r \approx -0,297$, `relative_humidity` и `wind_speed_ms` — $r \approx -0,280$. Это умеренно слабые связи и не основание для причинных выводов.

Практический блок 8.2. Правило IQR на примере скорости ветра

Следующий фрагмент не удаляет данные, а только маркирует необычные значения скорости ветра по стандартному правилу $1,5 \cdot \text{IQR}$. Сначала вычисляются квартильные характеристики и границы, затем строится булева маска всех наблюдений, лежащих вне интервала. В конце выводятся сами параметры правила и число кандидатов.

Метод `quantile(p)` возвращает квантиль заданного уровня. Квантиль уровня $p=0.25$ соответствует первому квартилю $Q1$, $p=0.75$ — третьему квартилю $Q3$. Межквартильный размах вычисляется как $iqr = q3 - q1$, после чего формируются эвристические границы $Q1 - 1.5 \cdot \text{IQR}$ и $Q3 + 1.5 \cdot \text{IQR}$.

Два сравнения с границами объединяются оператором `|` — поэлементным логическим «ИЛИ». Полученная маска содержит `True` для кандидатов на выброс, а `mask.sum()` считает их число.

```
q1 = df["wind_speed_ms"].quantile(0.25)
q3 = df["wind_speed_ms"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

mask = (
    (df["wind_speed_ms"] < lower) |
    (df["wind_speed_ms"] > upper)
)

print(f"Q1={q1:.3f}; Q3={q3:.3f}; IQR={iqr:.3f}")
print(f"Границы: [{lower:.3f}; {upper:.3f}]")
print("Кандидатов:", mask.sum())
```

Результат выполнения

```
Q1=0.110; Q3=0.660; IQR=0.550
Границы: [-0.715; 1.485]
Кандидатов: 3652 (13.69% строк)
```

Правило IQR помечает 3 652 наблюдения по скорости ветра. Такая доля слишком велика, чтобы автоматически считать эти строки ошибочными: скорее распределение имеет выраженный правый хвост. В реальном проекте решение об удалении возможно только после проверки источника и предметной интерпретации.

Такой порядок выполнения операций подчёркивает, что статистическая эвристика создаёт список для проверки, но не принимает решение об ошибочности наблюдений.

9. Визуальный разведочный анализ

График должен отвечать на конкретный аналитический вопрос. Столбчатая диаграмма удобна для сравнения категорий, гистограмма — для формы распределения одной числовой величины, график разброса — для взаимосвязи двух числовых переменных, временной ряд — для значения, упорядоченного по времени. Выбор типа графика определяется структурой данных, а не желанием использовать как можно больше визуализаций.

В таблице нет даты или времени события. Поэтому построение временного ряда «число пожаров по месяцам» на основе этого файла невозможно. Создавать искусственную дату или использовать индекс строки как время методологически неправильно.

Практический блок 9.1. Bar chart частоты типов событий

Перед построением графика частоты будут рассчитаны заново и отсортированы так, чтобы горизонтальные столбцы удобно сравнивались визуально. Полученная диаграмма предназначена не для поиска точных чисел — они уже известны из таблицы частот, — а для быстрой оценки относительных масштабов категорий и дисбаланса классов. Заголовок и подписи осей добавляются как обязательная часть аналитической визуализации.

Импорт библиотеки `import matplotlib.pyplot as plt` подключает модуль `pyplot` библиотеки `Matplotlib` под коротким именем `plt`.

Метод `value_counts()` вычисляет частоты категорий, `sort_values()` упорядочивает их, а `Series.plot(kind="barh")` использует интеграцию `Pandas` с `Matplotlib` для построения горизонтальной столбчатой диаграммы. Параметр `figsize` задаёт размер рисунка.

Методы объекта `Axes` `set_title()`, `set_xlabel()` и `set_ylabel()` оформляют график, `tight_layout()` корректирует поля, а `show()` выводит готовую фигуру.

```
import matplotlib.pyplot as plt

counts = df["type_name"].value_counts().sort_values()
ax = counts.plot(kind="barh", figsize=(8, 4.5))
ax.set_title("Количество наблюдений по типу события")
```

```
ax.set_xlabel("Число наблюдений")
ax.set_ylabel("Тип события")
plt.tight_layout()
plt.show()
```

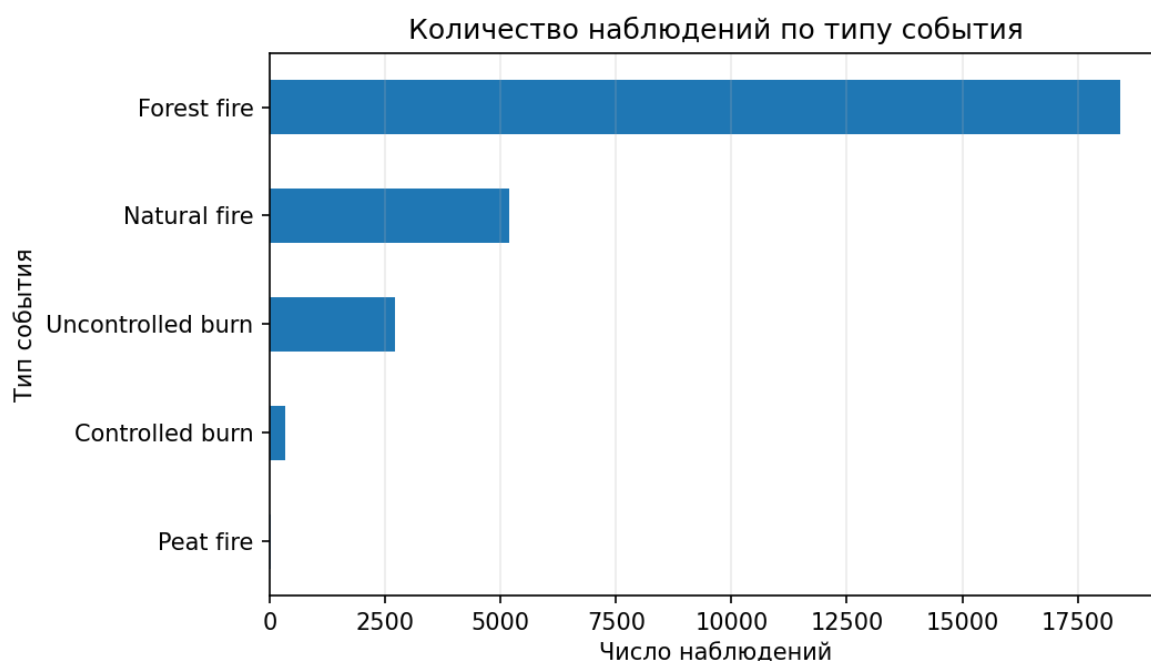


Рис. 1. Столбчатая диаграмма классов для оценки дисбаланса категорий

Визуально подтверждается сильный дисбаланс: Forest fire доминирует, Peat fire представлен минимально. При построении моделей обычная accuracy может оказаться вводящей в заблуждение, потому что доминирующий класс задаёт значительную долю правильных ответов даже у слабой модели.

Практический блок 9.2. Histogram: распределение температуры

Следующий график отвечает на вопрос о форме распределения temperature_c. Гистограмма позволит увидеть, где сосредоточена основная масса наблюдений, насколько распределение симметрично, имеются ли длинные хвосты или отдельные удалённые диапазоны. Число интервалов не следует воспринимать как единственно правильное значение, поскольку внешний вид гистограммы зависит от разбиения шкалы.

Функция plt.figure(figsize=...) создаёт отдельную фигуру.

Функция plt.hist() строит гистограмму числового признака: параметр bins=30 разбивает диапазон температуры на 30 интервалов, а высота каждого столбца показывает число наблюдений в соответствующем интервале; edgecolor задаёт цвет границ столбцов.

Функции `title()`, `xlabel()` и `ylabel()` задают контекст графика, `tight_layout()` уменьшает риск наложения элементов, `show()` завершает отображение.

```
plt.figure(figsize=(8, 4.5))
plt.hist(df["temperature_c"], bins=30, edgecolor="white")
plt.title("Распределение температуры")
plt.xlabel("Температура, °C")
plt.ylabel("Частота")
plt.tight_layout()
plt.show()
```

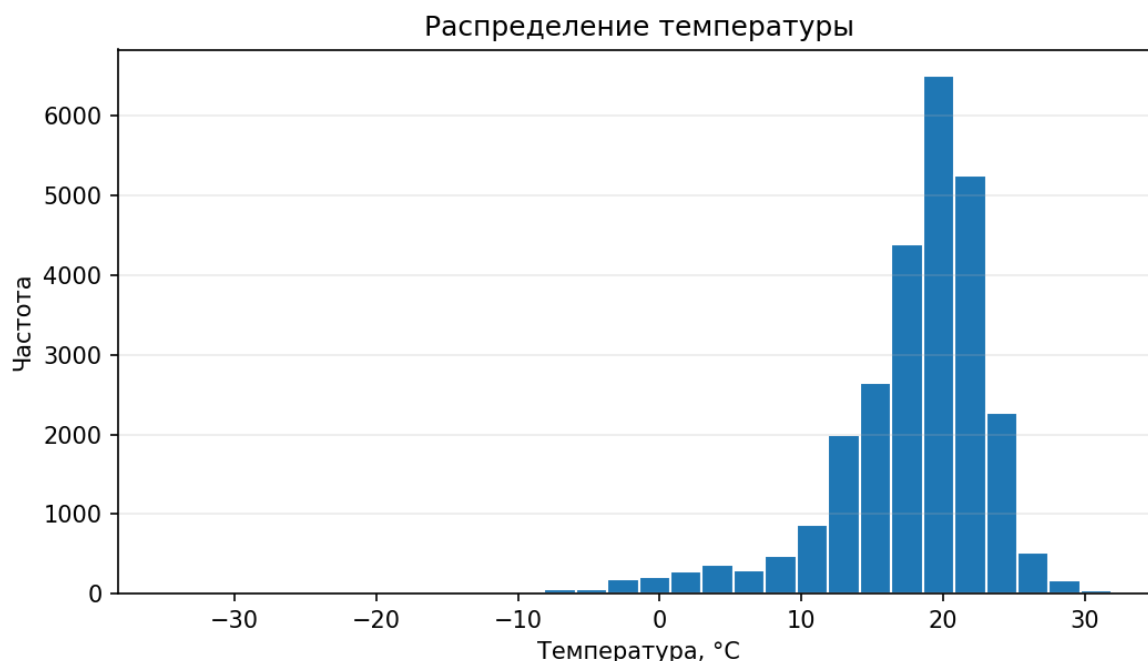


Рис. 2. Гистограмма для признака температуры

Основная масса наблюдений сосредоточена в положительном температурном диапазоне, но присутствует длинный левый хвост с отрицательными значениями до $-34,88$ °C. По одному графику нельзя объявлять эти значения ошибками. Необходима проверка происхождения наблюдений.

Практический блок 9.3. Scatter plot для температуры и влажности

Полный набор содержит более 26 тысяч точек, поэтому для учебной визуализации будет использована воспроизводимая подвыборка. Это уменьшает перегруженность графика, сохраняя общую структуру облака наблюдений. По оси X размещается температура, по оси Y — относительная влажность.

После построения нужно оценить направление, форму и плотность облака и сопоставить визуальное впечатление с числовым коэффициентом корреляции, полученным ранее.

Метод `df.sample(n=5000, random_state=42)` формирует случайную подвыборку из 5000 строк; фиксированный `random_state` делает выбор воспроизводимым при повторном запуске.

Функция `plt.scatter(x, y)` строит точечную диаграмму двух числовых переменных. Параметр `s` управляет размером маркеров, а `alpha` — прозрачностью; значение меньше 1 помогает увидеть плотные области при наложении точек. Остальные функции Matplotlib задают размер, заголовок, подписи осей и компоновку.

```
sample = df.sample(n=5000, random_state=42)

plt.figure(figsize=(8, 5))
plt.scatter(
    sample["temperature_c"],
    sample["relative_humidity"],
    s=10,
    alpha=0.35
)
plt.title("Температура и относительная влажность")
plt.xlabel("Температура, °C")
plt.ylabel("Относительная влажность")
plt.tight_layout()
plt.show()
```



Рис. 3. Парная точечная диаграмма разброса признаков влажности и температуры на подвыборке из 5000 строк

Точки образуют широкое облако без узкой линейной зависимости, что согласуется с корреляцией $r \approx -0,155$ между `temperature_c` и `relative_humidity`. При увеличении температуры в среднем наблюдается слабая тенденция к снижению влажности, однако вариативность остаётся высокой.

Практический блок 9.4. Пространственный scatter plot

В этом блоке категории событий будут нанесены на одну систему координат по долготе и широте. Группировка нужна не для агрегирования, а для раздельного построения слоёв с собственными подписями в легенде. Такая диаграмма позволяет увидеть пространственные кластеры и перекрытия типов событий. При интерпретации необходимо помнить, что координаты рисуются на обычных декартовых осях. Это разведочная визуализация геометрии точек, а не полноценная географическая карта.

Цикл `for name, part in df.groupby("type_name")` использует то, что итерация по объекту `GroupBy` последовательно возвращает пару «ключ группы, `DataFrame` этой группы».

Для каждой категории `plt.scatter()` добавляет на общую фигуру точки `lon/lat`.

Параметр `label=name` сохраняет название категории для легенды, а `plt.legend()` затем строит соответствующее обозначение. `s` и `alpha` регулируют размер и прозрачность маркеров.

```
plt.figure(figsize=(8, 5.5))
for name, part in df.groupby("type_name"):
    plt.scatter(
        part["lon"], part["lat"],
        s=6, alpha=0.35, label=name
    )

plt.xlabel("Долгота")
plt.ylabel("Широта")
plt.title("Пространственное распределение наблюдений")
plt.legend()
plt.tight_layout()
plt.show()
```

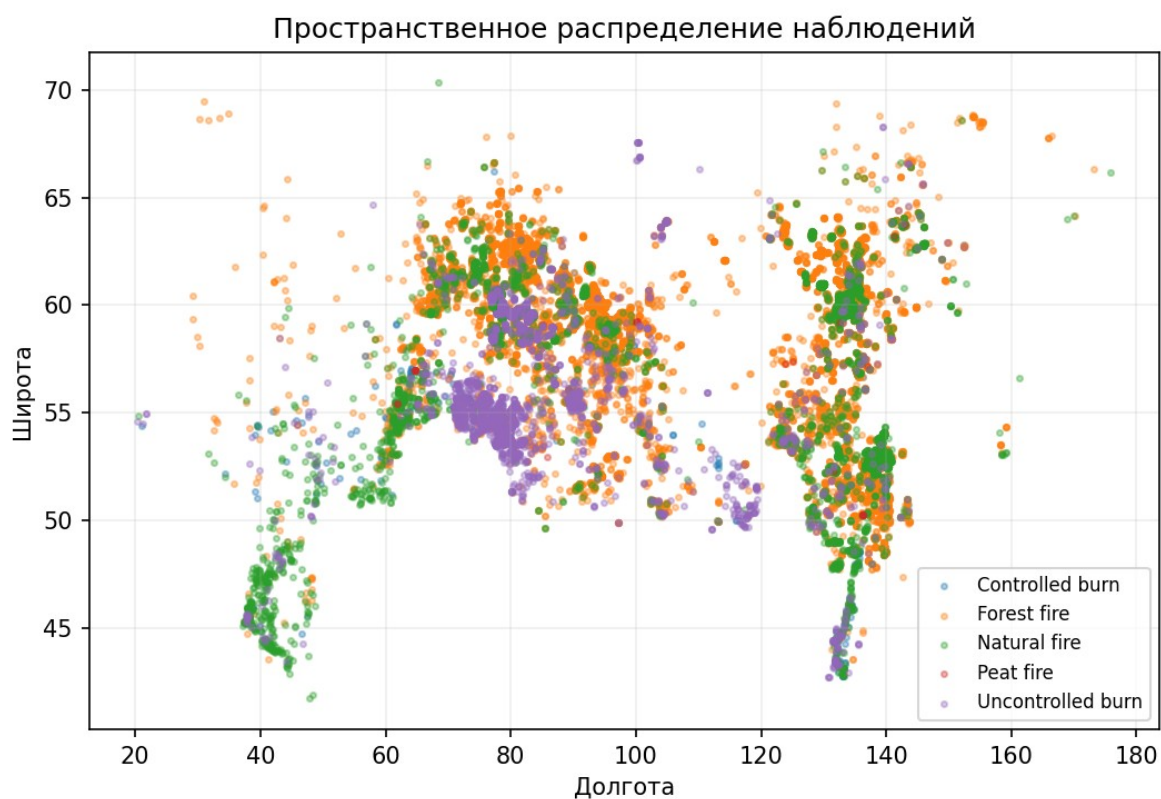


Рис. 4. Точечная диаграмма координат с цветом категории

Координаты занимают широкий диапазон долгот и широт, то есть набор пространственно неоднороден. Такой scatter plot показывает геометрию выборки, но не является картой: для географической интерпретации необходима картографическая проекция и геоданные границ регионов.

10. Экспорт CSV, JSON и Parquet

После анализа следует сохранить не сохраненное notebook-состояние, а явно сформированный итоговый экспортированный во внешний файл набор данных в виде DataFrame. CSV удобен для обмена, JSON — для полуструктурированного взаимодействия, Parquet — для последующей аналитической обработки. Для корректного сравнения форматов необходимо записывать одну и ту же таблицу и фиксировать параметры кодировки и сжатия.

Практический блок 10.1. Сохранение подготовленного DataFrame

Один и тот же подготовленный `clean_df` будет сохранён в трёх форматах в отдельный каталог `output`. Это принципиально важно для корректного сравнения: различаться должен формат хранения, а не содержимое таблицы. После выполнения блока появятся CSV для совместимого обмена, JSON Lines для построчного полуструктурированного представления и Parquet для аналитического чтения с сохранением схемы и колоночной организацией.

Объект `Path("output")` задаёт каталог результатов, а `OUT.mkdir(exist_ok=True)` создаёт его, не выдавая ошибку, если каталог уже существует. Оператор `/` у объектов `Path` формирует путь к файлу.

Метод `to_csv()` записывает таблицу в CSV; `index=False` исключает технический индекс, `encoding="utf-8"` задаёт кодировку.

Метод `to_json()` с `orient="records"` и `lines=True` формирует JSON Lines — один объект на строку; `force_ascii=False` сохраняет не-ASCII символы.

Метод `to_parquet()` использует движок `pyarrow` и сжатие `snappy` для записи колоночного Parquet.

```
from pathlib import Path

OUT = Path("output")
OUT.mkdir(exist_ok=True)

clean_df.to_csv(
    OUT / "FiresRu_clean.csv",
    index=False,
    encoding="utf-8"
)

clean_df.to_json(
    OUT / "FiresRu_clean.json",
    orient="records",
    lines=True,
    force_ascii=False
)
```

```
clean_df.to_parquet(
    OUT / "FiresRu_clean.parquet",
    index=False,
    engine="pyarrow",
    compression="snappy"
)
```

В исходном преподавательском окружении размеры исходной таблицы при записи без дополнительных столбцов составили примерно 1,46 MB для CSV и 4,69 MB для JSON Lines. Размер Parquet зависит от версии движка и параметров сжатия, поэтому его следует получить непосредственно в студенческом окружении и включить в отчёт как измеренный результат, а не как фиксированное число из методички.

Измерение размеров:

```
files = [
    OUT / "FiresRu_clean.csv",
    OUT / "FiresRu_clean.json",
    OUT / "FiresRu_clean.parquet"
]

for path in files:
    size_mib = path.stat().st_size / 1024**2
    print(f"{path.name:24s} {size_mib:7.2f} MiB")
```

Формат нельзя оценивать только по расширению. Для учебного отчёта необходимо сравнить размер файла, сохранение типов, скорость/удобство повторного чтения и возможность выборочного чтения столбцов. Parquet особенно важен как мост к следующему блоку курса — распределённой обработке в Spark.

Практический блок 10.2. Проверка round-trip

После экспорта данные будут прочитаны в два новых объекта `csv_back` и `parquet_back`. Сначала сравниваются размеры таблиц, затем исследуются типы, восстановленные из Parquet. Эта проверка нужна, чтобы убедиться, что файлы действительно пригодны для продолжения аналитического pipeline, а не просто были созданы на диске. Особое внимание следует уделить различию между текстовым CSV, где схема восстанавливается при чтении, и Parquet, где она хранится вместе с данными.

Термин round-trip означает цикл <записать данные, прочитать их обратно, проверить результат>.

Функция `pd.read_csv()` повторно загружает CSV и заново выводит типы из текстовых значений, тогда как `pd.read_parquet()` читает сохранённую Parquet-схему. Атрибут `shape` позволяет сопоставить число строк и столбцов с исходным DataFrame, а `dtypes` показывает тип каждого столбца в повторно загруженной таблице.

```
csv_back = pd.read_csv(OUT / "FiresRu_clean.csv")
parquet_back = pd.read_parquet(OUT / "FiresRu_clean.parquet")

print(csv_back.shape)
print(parquet_back.shape)
print(parquet_back.dtypes)
```

После повторного чтения необходимо убедиться, что число строк и столбцов совпадает с сохранённым DataFrame. Для CSV часть типов может быть восстановлена заново, тогда как Parquet хранит схему и обычно лучше сохраняет аналитические `dtypes`.

11. Итоговые выводы по набору FiresRu.csv

На основании выполненных операций можно сформулировать следующие выводы, относящиеся именно к предоставленному файлу, а не ко всей совокупности пожаров в России:

1. Таблица содержит 26 681 строку и 9 исходных признаков; пропущенных значений и полных дубликатов не обнаружено.
2. В данных представлены пять типов событий. Forest fire доминирует и занимает 69,02% строк; Peat fire представлен только 27 наблюдениями (0,10%).
3. Признак `type_id` является категориальным кодом, несмотря на целочисленный `dtype`. Использовать среднее значение `type_id` как содержательную статистику некорректно.
4. Преобразование `type_name` из `object` в `category` и `type_id` в `int8` уменьшает объём исходной таблицы в памяти примерно на 53,5% без потери информации.
5. Температура в наборе находится в диапазоне от $-34,88$ до $31,94$ °C; медиана $19,08$ °C. Ряд метеорологических признаков имеет асимметричные распределения.
6. Линейные корреляции между погодными признаками в целом невысоки; наиболее заметные по модулю коэффициенты в рассмотренной матрице находятся около 0,30.

7. Правило $1,5 \cdot \text{IQR}$ помечает значительную долю наблюдений скорости ветра как потенциальные выбросы, поэтому механическое удаление таких строк без предметной проверки недопустимо.

8. В файле отсутствует временной признак. Следовательно, анализ сезонности, динамики по месяцам или годам на основе только FiresRu.csv выполнить нельзя.

9. Координаты позволяют исследовать пространственную структуру, но корректная картографическая интерпретация требует GIS-инструментов, проекции и внешних границ территорий.

Первичный EDA не должен превращаться в последовательность вызовов функций. `head()`, `info()`, `describe()`, `groupby()`, `corr()` и графики — это инструменты проверки гипотез о структуре данных. Каждый вызов должен завершаться содержательным выводом и пониманием ограничений результата.

12. Самостоятельная работа. Анализ собственного датасета

После выполнения всех учебных блоков необходимо повторить ту же последовательность аналитики на собственном наборе данных. Датасет должен отличаться от FiresRu.csv и иметь понятный источник. Разрешены открытые государственные данные, репозитории исследовательских наборов, Kaggle/UC Irvine и иные источники при условии указания ссылки/происхождения и краткого описания лицензии или условий использования, если они опубликованы.

12.1. Минимальные требования к датасету

- не менее 500 строк и не менее 5 информативных столбцов;
- не менее двух числовых признаков;
- не менее одного категориального признака;
- желательно наличие пропусков, повторов или неоднозначных типов, чтобы предобработка была содержательной;
- если есть дата/время — привести её к `datetime` и построить временную агрегацию; если времени нет — прямо указать это ограничение и выбрать другой тип графика;
- датасет не должен содержать персональные или иные данные, использование которых нарушает требования конфиденциальности.

12.2. Обязательная последовательность работы

1. Создать отдельное виртуальное окружение или использовать базовое окружение курса; зафиксировать версии Python и библиотек.

2. Загрузить исходный файл. Если формат не CSV, объяснить особенности чтения. Для CSV явно проверить разделитель, кодировку и распознавание типов.
3. Сформировать «паспорт данных», который включает такие категории как: источник, предметная область, размер таблицы, перечень признаков и их предполагаемый смысл.
4. Выполнить базовые методы `head()`, `info()`, `describe(include="all")` или эквивалентный набор диагностик.
5. Посчитать пропуски, полные дубликаты и число уникальных значений по каждому столбцу. Обосновать каждое принятое действие с пропусками/дубликатами.
6. Проверить и при необходимости изменить `dtypes`. Сравнить память до и после хотя бы одной обоснованной оптимизации.
7. Создать минимум один вычисляемый признак с явной формулой или логическим правилом и объяснить его смысл.
8. Построить минимум один составной фильтр через `loc/query` и интерпретировать число полученных строк.
9. Выполнить минимум две группировки/агрегации, одна из которых должна использовать несколько агрегатных функций.
10. Исследовать не менее двух числовых признаков: описательная статистика, корреляция и один способ поиска потенциальных выбросов.
11. Построить не менее четырёх осмысленных визуализаций: категориальное сравнение, распределение, взаимосвязь и четвёртый график по структуре собственного набора. При наличии времени четвертым графиком должен быть временной ряд.
12. Экспортировать очищенный набор в CSV и Parquet; при возможности также JSON. Сравнить размеры файлов и проверить повторное чтение.
13. Сформулировать не менее пяти выводов о данных и не менее двух ограничений анализа.

12.3. Что должно быть в отчёте по самостоятельному анализу

Раздел	Содержание
1. Постановка задачи	Источник датасета, цель анализа, краткая предметная постановка.
2. Паспорт данных	Размер, признаки, <code>dtypes</code> , словарь полей.
3. Качество и предобработка	Пропуски, дубликаты, типы, преобразования и их обоснование.
4. EDA	Код, результаты функций, таблицы

Раздел	Содержание
	агрегатов, визуализации.
5. Интерпретация	Краткий вывод после каждого существенного блока.
6. Экспорт	CSV/Parquet, размеры, проверка повторного чтения.
7. Итог	Не менее 5 выводов и 2 ограничений исследования.

12.4. Что необходимо предоставить к сдаче работы

К сдаче работы необходимо привести:

1. Jupyter Notebook с выполненными учебными блоками по FiresRu.csv.
2. Отдельный Notebook/раздел отчёта по собственному датасету.
3. Краткие выводы после каждого аналитического блока и итоговые выводы по работе.

13. Контрольные вопросы

1. Почему dtype числового столбца не гарантирует, что признак является количественным?
2. Чем DataFrame отличается от NumPy ndarray с точки зрения схемы и типов?
3. Какие диагностические задачи решают head(), info() и describe() и почему одна функция не заменяет остальные?
4. Почему отсутствие пропусков и дубликатов ещё не означает отсутствие проблем качества данных?
5. В каких случаях category уменьшает память и почему?
6. Чем фильтрация отличается от группировки?
7. Что означает коэффициент корреляции около нуля? Какие выводы из него делать нельзя?
8. Почему правило IQR нельзя использовать как автоматический алгоритм удаления данных?
9. Как выбрать между histogram, bar chart, scatter plot и временным рядом?
10. Почему временной ряд нельзя строить по индексу строк, если индекс не является временем?
11. Какие преимущества Parquet даёт аналитическому pipeline по сравнению с CSV?
12. Почему воспроизводимый notebook должен корректно выполняться командой Run All после перезапуска ядра?